
1.10 Exercises

Reinforcement

- R-1.1 Write a short Java method, `inputAllBaseTypes`, that inputs a different value of each base type from the standard input device and prints it back to the standard output device.
- R-1.2 Suppose that we create an array *A* of `GameEntry` objects, which has an integer `scores` field, and we clone *A* and store the result in an array *B*. If we then immediately set `A[4].score` equal to 550, what is the score value of the `GameEntry` object referenced by `B[4]`?
- R-1.3 Write a short Java method, `isMultiple`, that takes two **long** values, *n* and *m*, and returns true if and only if *n* is a multiple of *m*, that is, $n = mi$ for some integer *i*.
- R-1.4 Write a short Java method, `isEven`, that takes an **int** *i* and returns true if and only if *i* is even. Your method cannot use the multiplication, modulus, or division operators, however.
- R-1.5 Write a short Java method that takes an integer *n* and returns the sum of all positive integers less than or equal to *n*.
- R-1.6 Write a short Java method that takes an integer *n* and returns the sum of all the odd positive integers less than or equal to *n*.
- R-1.7 Write a short Java method that takes an integer *n* and returns the sum of the squares of all positive integers less than or equal to *n*.
- R-1.8 Write a short Java method that counts the number of vowels in a given character string.
- R-1.9 Write a short Java method that uses a `StringBuilder` instance to remove all the punctuation from a string *s* storing a sentence, for example, transforming the string "Let's try, Mike!" to "Lets try Mike".
- R-1.10 Write a Java class, `Flower`, that has three instance variables of type **String**, **int**, and **float**, which respectively represent the name of the flower, its number of petals, and price. Your class must include a constructor method that initializes each variable to an appropriate value, and your class should include methods for setting the value of each type, and getting the value of each type.
- R-1.11 Modify the `CreditCard` class from Code Fragment 1.5 to include a method that updates the credit limit.
- R-1.12 Modify the `CreditCard` class from Code Fragment 1.5 so that it ignores any request to process a negative payment amount.
- R-1.13 Modify the declaration of the first **for** loop in the `main` method in Code Fragment 1.6 so that its charges will cause exactly one of the three credit cards to attempt to go over its credit limit. Which credit card is it?

Creativity

- C-1.14 Write a pseudocode description of a method that reverses an array of n integers, so that the numbers are listed in the opposite order than they were before, and compare this method to an equivalent Java method for doing the same thing.
- C-1.15 Write a pseudocode description of a method for finding the smallest and largest numbers in an array of integers and compare that to a Java method that would do the same thing.
- C-1.16 Write a short program that takes as input three integers, a , b , and c , from the Java console and determines if they can be used in a correct arithmetic formula (in the given order), like “ $a + b = c$,” “ $a = b - c$,” or “ $a * b = c$.”
- C-1.17 Write a short Java method that takes an array of **int** values and determines if there is a pair of distinct elements of the array whose product is even.
- C-1.18 The ***p-norm*** of a vector $v = (v_1, v_2, \dots, v_n)$ in n -dimensional space is defined as

$$\|v\| = \sqrt[p]{v_1^p + v_2^p + \dots + v_n^p}.$$

For the special case of $p = 2$, this results in the traditional ***Euclidean norm***, which represents the length of the vector. For example, the Euclidean norm of a two-dimensional vector with coordinates (4,3) has a Euclidean norm of $\sqrt{4^2 + 3^2} = \sqrt{16 + 9} = \sqrt{25} = 5$. Give an implementation of a method named `norm` such that `norm(v, p)` returns the p -norm value of v and `norm(v)` returns the Euclidean norm of v , where v is represented as an array of coordinates.

- C-1.19 Write a Java program that can take a positive integer greater than 2 as input and write out the number of times one must repeatedly divide this number by 2 before getting a value less than 2.
- C-1.20 Write a Java method that takes an array of **float** values and determines if all the numbers are different from each other (that is, they are distinct).
- C-1.21 Write a Java method that takes an array containing the set of all integers in the range 1 to 52 and shuffles it into random order. Your method should output each possible order with equal probability.
- C-1.22 Write a short Java program that outputs all possible strings formed by using the characters 'c', 'a', 't', 'd', 'o', and 'g' exactly once.
- C-1.23 Write a short Java program that takes two arrays a and b of length n storing **int** values, and returns the dot product of a and b . That is, it returns an array c of length n such that $c[i] = a[i] \cdot b[i]$, for $i = 0, \dots, n - 1$.
- C-1.24 Modify the `CreditCard` class from Code Fragment 1.5 so that `printSummary` becomes a *nonstatic* method, and modify the `main` method from Code Fragment 1.6 accordingly.
- C-1.25 Modify the `CreditCard` class to add a `toString()` method that returns a `String` representation of the card (rather than printing it to the console, as done by `printSummary`). Modify the `main` method from Code Fragment 1.6 accordingly to use the standard `println` command.

Projects

- P-1.26 Write a short Java program that takes all the lines input to standard input and writes them to standard output in reverse order. That is, each line is output in the correct order, but the ordering of the lines is reversed.
- P-1.27 Write a Java program that can simulate a simple calculator, using the Java console as the exclusive input and output device. That is, each input to the calculator, be it a number, like 12.34 or 1034, or an operator, like + or =, can be done on a separate line. After each such input, you should output to the Java console what would be displayed on your calculator.
- P-1.28 A common punishment for school children is to write out a sentence multiple times. Write a Java stand-alone program that will write out the following sentence one hundred times: “I will never spam my friends again.” Your program should number each of the sentences and it should make eight different random-looking typos.
- P-1.29 The *birthday paradox* says that the probability that two people in a room will have the same birthday is more than half, provided n , the number of people in the room, is more than 23. This property is not really a paradox, but many people find it surprising. Design a Java program that can test this paradox by a series of experiments on randomly generated birthdays, which test this paradox for $n = 5, 10, 15, 20, \dots, 100$.
- P-1.30 (*For those who know Java graphical user interface methods:*) Define a `GraphicalTest` class that tests the functionality of the `CreditCard` class from Code Fragment 1.5 using text fields and buttons.

Chapter Notes

For more detailed information about the Java programming language, we refer the reader to the Java website (<http://www.java.com>), as well as some of the fine books about Java, including the books by Arnold, Gosling and Holmes [8], Flanagan [33], and Horstmann and Cornell [47, 48].

